Applying Symbol Table in Real Life

Compiler Sessional Online

Problem Statement

We are going to apply symbol table in an interpreted language. Let's call it "Mickey Lang". An example code of the language is given below -

```
declare a
set a 2
{
    declare b
    set b 3
    declare c
    + c a b
    print c
}
```

Features

The features of the language is described below -

Keyword/Opcode	Description	Syntax
{ and }	Enter and Exit Scope	{}
`declare`	This declares a variable	`declare <variable>`
`set`	This sets the value of the variable.	`set <variable> <value>`
`+`	Adds two operands. The result is stored at the destination	`+ <dst> <operand1> <operand2>`
`print`	Prints the value of the variable	`print <var>`

Data Type

Mickey Lang only has 1 data type - **integers**.

Error Scenarios

There are several error scenarios that can happen -

- Declaring a variable more than once in the same scope
- Trying to assign to an undeclared variable (in a `set` operation or a `+` operation)

Skip any erroneous line altogether. Do NOT terminate the program

Examples

Sample Input

```
3
{
  declare a
  declare a
  set a 5
  {
    declare a
    set a 2
    + a a a
    + a a b
    + b a a
    set c 0
    print a
  }
  print a
}
```

Sample Output

```
        ScopeTable# 1 created
        ScopeTable# 2 created
        Inserted in ScopeTable# 2 at position 2, 1
> ERROR in line 4: 'a' already declared in this scope in line 3
> Set 'a' = 5 in ScopeTable# 2
        ScopeTable# 3 created
        Inserted in ScopeTable# 3 at position 2, 1
> Set 'a' = 2 in ScopeTable# 3
'a' found in ScopeTable# 3 at position 2, 1
'a' found in ScopeTable# 3 at position 2, 1
'a' found in ScopeTable# 3 at position 2, 1
> Set 'a' = 4 in ScopeTable 3
        'a' found in ScopeTable 3 at position 2, 1
> ERROR in line 11: 'b' not found in any of the ScopeTables
        'a' found in ScopeTable 3 at position 2, 1
        'a' found in ScopeTable 3 at position 2, 1
> ERROR in line 12: 'b' not found in any of the ScopeTables
> ERROR in line 13: 'c' not found in any of the ScopeTables
> 4
        ScopeTable 3 removed
> 5
        ScopeTable 2 removed
```

Explanation

- The first number indicates the **number of buckets** to use in your symbol table.
- The corresponding output log of each input line is colored in a matching fashion.
- **Hint:** you do not need to come up with complex parsing logic. Notice how the first word in each line determines what the syntax of that line is going to be. Use this to your advantage.

Boilerplate Code

```
ifstream code("code.txt");

int bucket_size;
code >> bucket_size;
int line = 0;

string opcode;
while (code >> opcode) {
  if (opcode == "{") {
    // handle scope creation
  }
  else if (opcode == "}") {
    // handle scope deletion
  }
  else if (opcode == "declare") {
    string variable;
    code >> variable;
    // handle variable declaration
  }
  else if (opcode == "set") {
    string variable;
    int value;
    code >> variable >> value;
    // handle variable assignment
  }
  else if (opcode == "+") {
    string dst, op1, op2;
    code >> dst >> op1 >> op2;
    // handle variable addition
  }
  else if (opcode == "print") {
    string variable;
    code >> variable;
    // handle variable printing
  }
  line++;
}
```

Marks Distribution

Task	**Marks**
Enter and Exit Scope	1
`declare` Operation • Redeclaration error handling	2
`set` Operation • Assignment to undeclared variable error handling	3
Add (`+`) Operation • Assignment to undeclared variable error handling	3
`print`	1
Total	**10**